



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 7

Team No.: 5

Project Title: Multithreaded-Linux-Application-Using-POSIX-Threads-and-Mutexes

Team Members

Roll Number	Student Name	Signature
2500031613	U.Moksha	
2520030408	Jamuna	
2520030416	Keerthi	

1. Abstract

The proposed project focuses on developing a multithreaded Linux application using POSIX threads (pthreads) and mutexes to demonstrate concurrent execution and safe resource sharing. In modern operating systems, multiple tasks often need to execute simultaneously while accessing shared resources. Without proper synchronization, concurrent access can lead to race conditions, inconsistent data, and unexpected program behavior. This project explores important Operating Systems and System Programming concepts such as processes and threads, concurrency, parallel execution, shared memory, thread creation and termination, synchronization, critical sections, and mutual exclusion. The application will be implemented in Linux using the POSIX Threads library, where multiple threads will perform tasks concurrently and mutex locks will be used to control access to shared resources. The proposed solution will demonstrate how threads can improve task execution while synchronization mechanisms ensure data consistency and prevent conflicts between threads. The application will also allow observation of thread behavior and synchronization during execution. The expected outcome is a functional Linux-based multithreaded application that clearly demonstrates thread management and mutex-based synchronization, providing practical understanding of how operating systems handle concurrent tasks and shared resources.

2. Problem Statement

The project addresses the problem of multiple threads accessing shared resources simultaneously, which can cause race conditions and data inconsistency. It uses POSIX threads and mutexes in Linux to demonstrate concurrent execution and safe resource sharing. This is relevant to Operating Systems and Systems Programming because thread management, synchronization, critical sections, and mutual exclusion are essential concepts for developing reliable multithreaded applications.

3.Objectives

- To create and manage multiple threads using **POSIX Threads (pthreads)**.
- To demonstrate **concurrent execution** of multiple tasks.
- To implement **mutex locks** for thread synchronization.
- To prevent **race conditions and data inconsistency**.
- To demonstrate safe access to **shared resources**.
- To understand **critical sections** in multithreaded programs.
- To study **thread creation, execution, and termination** in Linux.
- To compare program behavior with and without **synchronization**.
- To understand the role of **concurrency in Operating Systems**.
- To develop a functional **Linux-based multithreaded application**.

4. Proposed Methodology

The project will be designed and implemented in Linux using the **C programming language** and the **POSIX Threads (pthreads) library**. Multiple threads will be created to perform tasks concurrently, while mutex locks will be used to control access to shared resources and prevent race conditions. The application will first initialize the required resources, create multiple threads, and assign tasks to each thread. Threads will execute their tasks simultaneously and use mutexes whenever they access shared data. After completing their tasks, the threads will terminate and the main program will collect their results using thread joining. The final output will demonstrate how multithreading and synchronization work together to ensure safe and consistent execution.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
POSIX Threads (pthread)	Create and manage multiple threads for concurrent execution.
pthread_create()	Creates and starts individual threads.
pthread_join()	Waits for threads to complete before the main program continues.
Mutex (pthread_mutex_t)	Provides mutual exclusion when accessing shared resources.
pthread_mutex_lock() / pthread_mutex_unlock()	Locks and releases the critical section to prevent race conditions.
Critical Section	Protects shared data from simultaneous access by multiple threads.
Thread Synchronization	Ensures threads access shared resources safely and maintain data consistency.

6. Individual contribution

Roll Number	Student Name	Individual Responsibility
2520031613	U.Moksha	• Read information from /proc • Use commands like ps and top • Develop the process-monitoring module • Display PID, process state,CPU and memory usage • Monitor running processes
2520030408	Jamuna	• CPU information and usage • RAM/memory usage • Disk usage • System uptime • OS/kernel information • Develop the system-resource module
2520030416	Keerthi	• Create the main menu/interface • User input and navigation • Connect both modules • Add refresh/exit options • Error handling • Combine and test the complete project

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating environment for developing and running the system monitoring application.
C / C++	Used to implement the monitoring modules and interact with Linux system resources.
GCC / G++ Compiler	Used to compile and build the C/C++ source code.
Linux /proc Filesystem	Used to obtain real-time information about processes, CPU, memory, and system resources.
Git / GitHub	Used for version control, collaboration, and managing the project source code.

8. Expected Outcome

The project is expected to provide a Linux-based system monitoring application that displays real-time information about running processes and system resources. It will show details such as process ID, process state, CPU and memory usage, along with overall CPU, RAM, disk usage, and system information. The application will provide a simple menu-driven interface, allow users to view and refresh monitoring information, and help users understand the current performance and resource utilization of the Linux system.

9. GitHub Repository

GitHub Repository Link: <https://github.com/Jamuna408/Multithreaded-Linux-Application-Using-POSIX-Threads-and-Mutexes.git>

Repository Created: yes

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: K Hema

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date: 26-08-2026

Faculty Signature:

